SYSTEM DESIGN MINI PROJECT · COOKBOOK

Edge CDN Cache with Origin Shielding

A step-by-step recipe book for building a tiered CDN with request collapsing, failure handling and live metrics, for a simulated newspaper platform.

Python 3.12 · FastAPI · aiohttp · Redis · MinIO · Pillow · Docker Compose · Render

Repository: ShreyashSingh857/SSD-MiniProject

19 recipes · 6 parts

Contents

How to use this cookbook

The finished dish

Kitchen equipment (prerequisites)

Part 1. Design before cooking

Recipe 1: Decide the architecture and the vocabulary

Part 2. The shared pantry: shared/cdncore

Recipe 2: Lay out the repository

Recipe 3: Configuration (config.py)

Recipe 4: The cache store (cache.py)

Recipe 5: Metrics (metrics.py)

Recipe 6: The node admin kit (node.py)

Recipe 7: Calling the next layer (upstream.py)

Part 3. The services

Recipe 8: Generate the newspaper (services/origin/content/generator.py)

Recipe 9: The object store (services/minio)

Recipe 10: The origin service (services/origin/origin_app.py)

Recipe 11: The origin shield (services/shield/shield_app.py)

Recipe 12: The edge cache (services/edge/edge_app.py)

Recipe 13: The router (services/router/router_app.py)

Recipe 14: The web pages (services/web and content/static/app.js)

Recipe 15: Benchmarks (services/router/bench.py)

Part 4. Packaging and serving

Recipe 16: Dockerfiles and start scripts

Recipe 17: Run everything with Docker Compose

Recipe 18: Tests

Part 5. Serving it to the world

Recipe 19: Deploy on Render

Part 6. Lessons from the kitchen

Ideas for a second edition

How to use this cookbook

This book explains how the **Edge CDN Cache with Origin Shielding** project was built, one recipe at a time. Each recipe has the same parts:

- **Goal**: what you're making.
- **Files**: where it lives in the repository.
- **Method**: the steps, with the real code from the project.
- **Why it's built this way**: the design reasoning, including mistakes that were fixed.
- **Taste test**: a command or check that proves the step works.

Follow the recipes in order and you'll rebuild the whole system: tiered caching, request collapsing, failure handling, metrics, a newspaper website, a monitoring dashboard, benchmarks, tests, Docker packaging and deployment on Render.

The code excerpts are copied from the repository (`ShreyashSingh857/SSD-MiniProject`). Long functions are shortened where marked `# ...`, and a few long lines are re-wrapped to fit the page; the full version is always in the named file.

The finished dish

A miniature content delivery network for a simulated newspaper, *The Daily Edge*:

```
Browser ─► Web (static pages)
   │
   └─► Router ──round-robin + health checks──► Edge 1 │ Edge 2 │ Edge 3     (each: FastAPI + own Redis)
                                                  │ miss
                                                  ▼
                                Origin Shield  (FastAPI + shared Redis)
                                ├─ request collapser: one origin fetch per object
                                ├─ bounded origin pool (4 concurrent fetches)
                                └─ circuit breaker
                                                  │ miss
                                                  ▼
                                Origin service ─► MinIO   (articles, images, thumbnails, static files)
```

Kitchen equipment (prerequisites)

Tool	Version used	Used for
Python	3.12 (inside containers)	All services
Docker Desktop with Docker Compose	29.x	Running every service as a container
Git and a GitHub account	any	Version control and Render deployment
A browser	any modern one	Newspaper and dashboard

Python libraries (installed per service from its own `requirements.txt`):

Library	Purpose
`fastapi`, `uvicorn[standard]`	HTTP services (uvicorn's "standard" extra adds the fast `uvloop` and `httptools`)
`aiohttp`	HTTP client for calls between services
`redis`	Async Redis client (edge and shield only)
`minio`	S3 client (origin only)
`pillow`	Rendering the newspaper's JPEG images (origin only)
`pytest`	Tests (development only, `requirements-dev.txt`)

Part 1. Design before cooking

Recipe 1: Decide the architecture and the vocabulary

Goal: fix the moving parts, the request paths and the names everything will use, before any code is written.

The request paths

Every request starts at the router and stops at the first layer that has the object:

Path	Route	Typical latency locally
Edge hit	User → Router → Edge → User	~4–6 ms
Shield hit	User → Router → Edge → Shield → Edge (stores it) → User	~5–8 ms
Origin miss	User → Router → Edge → Shield → Origin → MinIO, then back, stored at shield and edge	~150 ms (120 ms is artificial delay)

Object IDs

Every cacheable thing has an ID of the form `type:name`. The type picks the MinIO folder and the cache lifetime:

Object ID	Stored in MinIO as	Content
`article:world-013`	`article/world-013`	Article JSON
`image:world-013`	`image/world-013`	1200×675 JPEG
`thumbnail:world-013`	`thumbnail/world-013`	400×225 JPEG
`static:homepage.css`	`static/homepage.css`	CSS, JS or SVG
`index:latest`	`index/latest`	Homepage article list (JSON)

Response headers

Each response says where it was served from. These labels drive the inspector, the dashboard and the benchmarks:

`X-Cache` value	Meaning
`HIT-EDGE`	Served from the edge's own Redis
`HIT-SHIELD`	Edge missed, shield had it
`MISS-ORIGIN`	Both missed; fetched from MinIO
`MISS-COLLAPSED`	Waited on another request's in-flight origin fetch
`MISS-ORIGIN-BYPASS`	Shield was unreachable; the edge went straight to the origin
`STALE-EDGE` / `STALE-SHIELD`	Every upstream failed; an expired copy was served
`ERROR`, `TIMEOUT`, `NO-EDGE`	Nothing could serve it

The services also set `X-Served-By` (which node answered), `X-Content-Class` (which TTL class), and the router adds `X-Request-Id` and `X-Latency-ms`.

Why it's built this way

- **The newspaper is the workload, not the subject.** The project document says the CDN layer is what's being demonstrated, so the content is generated and fictional (Recipe 8).
- **An origin service in front of MinIO.** MinIO isn't our code, so a small FastAPI service sits in front of it. It adds the artificial delay, counts origin requests and can be "failed".
- **Every layer is its own service.** That makes real failures possible (stop a container) and maps directly onto separate deployments later.

Part 2. The shared pantry: `shared/cdncore`

Five small modules are used by several services. They're copied into each Python image at build time.

Recipe 2: Lay out the repository

Goal: a folder structure where each microservice can be built and deployed on its own.

```
shared/
  cdncore/            config.py, cache.py, metrics.py, node.py, upstream.py
  start-with-redis.sh start script for the cache services
services/
  router/   router_app.py, bench.py, Dockerfile, requirements.txt
  edge/     edge_app.py, Dockerfile, requirements.txt
  shield/   shield_app.py, Dockerfile, requirements.txt
  origin/   origin_app.py, content/ (generator + static files), Dockerfile, requirements.txt
  minio/    Dockerfile, start.sh
  web/      index.html, dashboard.html, config.js
tests/      conftest.py, test_units.py, test_e2e.py
docker-compose.yml, docker-compose.render-sim.yml, render.yaml
```

Method:

1. Create the folders above.
2. Add `.gitattributes` so files always use Unix line endings. Windows would otherwise check out shell scripts with CRLF line endings, which breaks them inside Linux containers:

   ```
   * text=auto eol=lf
   *.jpg binary
   *.png binary
   ```

3. Add `.gitignore` (`__pycache__/` , `*.pyc` , `.venv/` , `.pytest_cache/`) and `.dockerignore` (`.git` , `tests` , `docs` , `*.md` , caches) so images stay small.

Why it's built this way: service code is named `edge_app.py` , `shield_app.py` and so on rather than `app.py` . Unique names let the tests import several services into one Python process.

Recipe 3: Configuration (`config.py`)

Goal: one place for settings. Some come from environment variables at startup; others can be changed while the system runs.

Method:

1. Read static settings from the environment, with defaults that work on one local network:

```
NODE_ID = _env("NODE_ID", "node")
PORT = int(_env("PORT", "8000"))
REDIS_URL = _env("REDIS_URL", "redis://localhost:6379/0")

# Shared secret on every hop between services. Empty disables the check (local development).
INTERNAL_TOKEN = _env("INTERNAL_TOKEN", "")
# Protects the router's public POST control API (failures, flush, config, publish, benchmarks).
CONTROL_TOKEN = _env("CONTROL_TOKEN", "")

# Defaults suit one local network. Hosts where services sleep need much longer values.
CONNECT_TIMEOUT_S = float(_env("CONNECT_TIMEOUT_S", "1.5"))
UPSTREAM_TIMEOUT_S = float(_env("UPSTREAM_TIMEOUT_S", "5"))
```

2. Parse the edge list, given as `name=url` pairs:

```
def parse_edges(spec: str = EDGE_URLS) -> dict[str, str]:
    edges: dict[str, str] = {}
    for part in spec.split(","):
        part = part.strip()
        if part:
            name, url = part.split("=", 1)
            edges[name.strip()] = url.strip().rstrip("/")
    return edges
```

3. Define a TTL (cache lifetime) per content class. Breaking news changes fast; CSS barely changes:

```
DEFAULT_TTLS = {
    "breaking": 20, "index": 10, "popular": 120, "regular": 300,
    "archive": 900, "image": 1800, "thumbnail": 1800, "static": 3600,
}
```

4. Hold everything changeable at runtime in a dataclass, `RuntimeConfig`. The shield keeps objects twice as long as the edges (`shield_ttl_factor`):

```
@dataclass
class RuntimeConfig:
    ttls: dict[str, int] = field(default_factory=lambda: dict(DEFAULT_TTLS))
    shield_ttl_factor: float = 2.0
    # How long an expired object is kept around to be served if upstream layers fail.
    stale_grace_seconds: int = 3600
    collapsing: bool = True
    origin_delay_ms: int = 120
    origin_jitter_ms: int = 40
    upstream_timeout_s: float = UPSTREAM_TIMEOUT_S

    def ttl_for(self, content_class: str, layer: str = "edge") -> int:
        ttl = self.ttls.get(content_class, self.ttls["regular"])
        if layer == "shield":
            ttl = int(ttl * self.shield_ttl_factor)
        return max(1, ttl)
```

5. Give it an `update()` method that accepts JSON from the dashboard. Booleans need special care, because `bool("false")` is `True` in Python:

```
    if isinstance(current, bool) and not isinstance(value, bool):
        value = str(value).lower() in ("1", "true", "yes", "on")
    setattr(self, key, type(current)(value))
```

Taste test: `tests/test_units.py::test_config_update_parses_booleans_and_ttls` sends `{"collapsing": "false"}` and checks the result is really `False`.

Recipe 4: The cache store (`cache.py`)

Goal: a Redis-backed store that keeps each object's body, type and expiry together, and can still return an expired copy when everything upstream is failing.

Method:

1. Store each object as a Redis hash under `obj:<object_id>`, with fields for the body, content type, content class, creation time and logical expiry time.
2. Give the **Redis key** a longer life than the **logical TTL**. The gap (the *grace period*) is what makes stale serving possible:

```
async def set(self, object_id, body, content_type, content_class, ttl, grace):
    now = time.time()
    key = KEY_PREFIX + object_id
    async with self.redis.pipeline(transaction=True) as pipe:
        pipe.delete(key)
        pipe.hset(key, mapping={
            "body": body, "ctype": content_type, "cls": content_class,
            "created": now, "expires": now + ttl,
        })
        # The Redis key outlives the logical TTL by `grace` so stale copies can be served on failure.
        pipe.expire(key, ttl + grace)
        await pipe.execute()
```

3. `get()` always returns what it finds, fresh or not. The caller checks `cached.fresh`:

```
@property
def fresh(self) -> bool:
    return time.time() < self.expires_at
```

4. Add `delete()`, `flush()` (uses `SCAN`, never `KEYS`, so Redis isn't blocked) and `status()` (key counts, memory used, Redis's own evicted and expired counters) for the dashboard.
5. Create the client with short timeouts, so a dead Redis doesn't hang requests: `aioredis.from_url(redis_url, socket_timeout=1.0, socket_connect_timeout=1.0)`.

Why it's built this way:

- This is **cache-aside**: the service checks the cache, and on a miss it fetches from the next layer and stores the result. The cache never fetches on its own.
- **Eviction** is Redis's job. Each Redis runs with `--maxmemory` and `--maxmemory-policy allkeys-lru`, so memory pressure removes the least recently used objects.

Recipe 5: Metrics (`metrics.py`)

Goal: counters and latency percentiles for every service, cheap enough to update on every request.

Method:

1. Keep a `Counter` for named events (`hits`, `misses`, `collapsed`, ...), a second `Counter` for per-key counts (for example how often each object reached the origin), and a bounded `deque` of the last 5,000 `(timestamp, latency_ms)` samples.
2. Compute percentiles by sorting the window:

```
def percentile(values: list[float], pct: float) -> float:
    if not values:
        return 0.0
    ordered = sorted(values)
    k = min(len(ordered) - 1, max(0, int(round(pct / 100 * (len(ordered) - 1)))))
    return ordered[k]
```

3. Throughput is the number of samples in the last 10 seconds divided by 10 (or by uptime, if shorter).
4. `snapshot()` returns counters, avg/p50/p95/p99 latency, top keys and throughput as JSON.

Why it's built this way: a fixed window keeps memory constant and sorting 5,000 numbers is fast. A production system would use a histogram library or Prometheus, which was out of scope.

Recipe 6: The node admin kit (`node.py`)

Goal: give every internal service the same health, metrics, config, flush and failure endpoints without repeating code.

Method:

1. A `NodeState` object holds the node's ID, role, cache store, `failed` flag, metrics and runtime config.
2. **Failure simulation as middleware.** When `failed` is set, the node answers everything with 503 and a header `X-Node-Down: 1`, except `/healthz` and the `/admin` API that restores it:

```
class FailureGate:
    async def __call__(self, scope, receive, send):
        if scope["type"] == "http" and self.state.failed and scope["path"] != "/healthz" \
                and not scope["path"].startswith("/admin"):
            await down_response(self.state)(scope, receive, send)
            return
        await self.app(scope, receive, send)
```

3. **Authentication between services as middleware.** When `INTERNAL_TOKEN` is set, every request except `/health` and `/healthz` must carry it. `hmac.compare_digest` avoids timing leaks:

```python
class InternalAuth:
    async def __call__(self, scope, receive, send):
        if (scope["type"] == "http" and config.INTERNAL_TOKEN
                and scope["path"] not in ("/health", "/healthz")):
            sent = dict(scope["headers"]).get(b"x-internal-token", b"").decode("latin-1")
            if not hmac.compare_digest(sent, config.INTERNAL_TOKEN):
                denied = JSONResponse({"error": "invalid internal token"}, status_code=401)
                await denied(scope, receive, send)
                return
        await self.app(scope, receive, send)
```

4. `install_admin(app, state)` adds both middlewares and these endpoints:

Endpoint	Purpose
`GET /healthz`	Liveness for the hosting platform. Stays 200 during a simulated failure
`GET /health`	Readiness, used by the router. Returns 503 while "failed"
`POST /admin/failure`	`{"down": true/false}`
`GET /admin/metrics`, `POST /admin/metrics/reset`	Metrics
`GET/POST /admin/config`	Runtime config
`GET /admin/status`	Cache contents plus service-specific extras
`POST /admin/flush`	Empty the cache, or one object

Why it's built this way:

- **Pure ASGI middleware, not `@app.middleware("http")`.** Starlette's decorator style (`BaseHTTPMiddleware`) adds noticeable overhead per request. These classes add almost none.
- **Two health endpoints.** Render restarts a service whose health check fails. If Render checked `/health`, a simulated failure would get the node restarted and end the demo. So Render uses `/healthz` and the router uses `/health`.
- `add_middleware` wraps outward: the last one added runs first, so authentication runs before the failure gate.

Recipe 7: Calling the next layer (`upstream.py`)

Goal: one HTTP helper that every hop uses, and that classifies failures correctly. Most of the failure handling depends on this classification.

Method:

1. Use **aiohttp** with one shared session per service. The session sends the internal token on every request and keeps DNS cache entries for only 2 seconds:

```
def new_session(limit: int = 200) -> aiohttp.ClientSession:
    headers = {"X-Internal-Token": config.INTERNAL_TOKEN} if config.INTERNAL_TOKEN else None
    connector = aiohttp.TCPConnector(limit=limit, keepalive_timeout=30, ttl_dns_cache=2)
    return aiohttp.ClientSession(headers=headers, connector=connector)
```

2. Use two timeouts: a short **connect** timeout (is the node there at all?) and a longer **total** timeout (is it answering?):

```
client_timeout = aiohttp.ClientTimeout(total=timeout, connect=min(config.CONNECT_TIMEOUT_S, timeout))
```

3. Turn every outcome into one of three results: success, `NotFound`, or `UpstreamError` with a `node_down` flag that says whether the node is *dead* or just *slow or failing*:

```
async def fetch(session, url, timeout) -> Fetched:
    try:
        status, headers, body = await get_raw(session, url, timeout)
    except aiohttp.ConnectionTimeoutError as exc:
        raise UpstreamError(f"connect timeout calling {url}", node_down=True) from exc
    except asyncio.TimeoutError as exc:
        # Slow, not dead: callers should not treat a read timeout as the node being down.
        raise UpstreamError(f"timeout calling {url}", node_down=False) from exc
    except aiohttp.ClientError as exc:
        raise UpstreamError(f"{exc.__class__.__name__} calling {url}", node_down=True) from exc
    if status == 404:
        raise NotFound(url)
    if status >= 400:
        # 502/504 come from the hosting proxy, never from our services: the node itself is unavailable.
        raise UpstreamError(f"HTTP {status} from {url}",
                            node_down=headers.get("x-node-down") == "1" or status in (502, 504))
    return Fetched(body=body, content_type=headers.get("content-type", "application/octet-stream"),
                   content_class=headers.get("x-content-class", "regular"), headers=headers)
```

Why it's built this way (lessons learned):

- **aiohttp, not httpx.** The first version used httpx. Measured inside Docker, the router forwarding through httpx managed only about 390 requests/s, while an edge alone served about 1,650. Switching every hop to aiohttp raised the router to about 1,860 requests/s.
- **"Slow" is not "dead".** An early version treated every timeout as a dead node. When MinIO was stopped, the origin waited, the shield timed out, the edge timed out and the router finally marked all three healthy edges as dead. Separating connection failures from read timeouts fixed it.
- **Short connect timeout.** A stopped container's hostname took 4–8 s to fail DNS lookup. The 1.5 s connect timeout cut failover time from 7.9 s to about 1.5 s.

Part 3. The services

Recipe 8: Generate the newspaper (`services/origin/content/generator.py`)

Goal: realistic, repeatable content, with no copyright issues and no external API.

Method:

1. **Word pools.** Define fictional cities, people, teams, companies and authors, plus headline templates per section (world, politics, business, technology, sports, science, health, culture), breaking-news templates and paragraph templates with slots like `{city}`, `{person}` and `{pct}`.
2. **An article model:**

```python
@dataclass
class Article:
    article_id: str
    title: str
    section: str
    category: str  # content class used for TTL: breaking | popular | regular | archive
    summary: str
    content: list[str]
    author: str
    created_at: str
    image_path: str
    thumbnail_path: str
    read_minutes: int
    tags: list[str] = field(default_factory=list)
```

3. **A seeded random generator** (`random.Random(5061)`), so the same articles come out every time. The tests and demos can then rely on IDs like `world-013`. `generate_articles(60)` produces:
   - 5 breaking-news articles from the last 2 hours (`breaking-news-001` to `-005`)
   - 15 popular articles from the last 2 days
   - 25 regular articles from the last month
   - 15 archive articles, 1 month to 2.5 years old
4. **Images with Pillow.** `render_image()` draws an abstract "press photo": a vertical gradient in the section's colour, blurred circles of light, a city skyline with lit windows, and a black label with the section name (plus "BREAKING"). Each article gets a 1200×675 hero image and a 400×225 thumbnail. The article ID seeds the drawing, so images are repeatable too.
5. **Static assets.** `homepage.css`, `app.js` and `logo.svg` live in `content/static/` and are loaded as `static:` objects. The site's own CSS and JS are therefore delivered through the CDN.
6. **The homepage index.** `build_index()` writes `index:latest`: a list of article "cards" (every field except the body).
7. **A seed version.** `seed_version()` hashes the generator version and the static files. The origin compares it with a marker in MinIO, so content is reseeded only when it changes.
8. **Pre-rendered images.** `prerender_images()` runs at Docker build time and saves every JPEG into `/app/image-cache`. `cached_image()` uses those files at startup and renders only when missing.

Why it's built this way: rendering 120 blurred JPEGs takes about 3 s on a laptop, but would take minutes on a 0.1-CPU hosted instance, every time it starts. Doing it once at build time removes that from startup.

Taste test: `test_generator_is_deterministic_and_complete` generates the articles twice and compares them, and checks a rendered image starts with the JPEG signature `FF D8`.

Recipe 9: The object store (`services/minio`)

Goal: run MinIO as its own service, on whatever port the host assigns.

Method:

1. Base the image on `pgsty/minio`, pinned to `RELEASE.2026-06-18T00-00-00Z`.
2. Add a start script that maps our variable names to MinIO's and turns off the web console when only one port is available:

```
export MINIO_ROOT_USER="${MINIO_ROOT_USER:-$MINIO_ACCESS_KEY}"
export MINIO_ROOT_PASSWORD="${MINIO_ROOT_PASSWORD:-$MINIO_SECRET_KEY}"
mkdir -p /data
if [ "$MINIO_BROWSER" = "off" ]; then
  exec minio server /data --address ":${PORT:-9000}" --quiet
fi
exec minio server /data --address ":${PORT:-9000}" --console-address ":9001"
```

Why it's built this way:

- **The image name.** In 2026 the upstream MinIO project was archived and its official Docker images (`minio/minio`, `quay.io/minio/minio`) and binary downloads were withdrawn. `pgsty/minio` is a maintained community build of the same server with the same S3 API.
- **Console only locally.** Render exposes one HTTP port per service, and the console needs a second.
- MinIO's access key must not contain `=` or `,`, so it's a fixed name (`cdnadmin`). Only the secret key is random.

Taste test: log in to the console at `http://localhost:9001` with `cdnadmin` / `cdnadmin-secret` and open the `newspaper` bucket.

Recipe 10: The origin service (`services/origin/origin_app.py`)

Goal: the single source of truth. It reads from MinIO, simulates distance, loads content on startup and publishes new articles.

Method:

1. **Create a MinIO client that fails fast.** By default urllib3 retries with long backoff, which caused the cascading timeouts described in Recipe 7:

```
http_client = urllib3.PoolManager(
    timeout=urllib3.Timeout(connect=config.MINIO_CONNECT_TIMEOUT_S, read=config.MINIO_READ_TIMEOUT_S),
    retries=urllib3.Retry(total=1, backoff_factor=0.1),
    maxsize=32)
client = Minio(config.MINIO_ENDPOINT, access_key=config.MINIO_ACCESS_KEY,
               secret_key=config.MINIO_SECRET_KEY, secure=config.MINIO_SECURE,
               region="us-east-1", http_client=http_client)
```

Passing `region` skips an extra bucket-location request to MinIO.

2. **Validate object IDs** before touching storage, which also blocks path tricks like `..`:

```
def storage_key(object_id: str) -> str | None:
    kind, _, name = object_id.partition(":")
    if kind not in OBJECT_TYPES or not name or ".." in name:
        return None
    return f"{kind}/{name}"
```

3. **Seed in the background.** On startup, a task retries until MinIO answers, creates the bucket and uploads every generated object, unless the seed marker already matches. Each object's content class is saved as MinIO metadata (`x-amz-meta-content-class`), so every later layer knows which TTL applies.
4. **Serve objects** with an artificial delay of 120 ms plus 0–40 ms of random jitter, standing in for a far-away origin. The MinIO call runs in a thread (`asyncio.to_thread`) because the MinIO client isn't async. Every request is counted as an *origin request*.
5. **Reseed after a wipe.** If MinIO answers "no such key" and the seed marker is also missing, the storage has been emptied (on Render that happens whenever MinIO sleeps). The origin returns 503 once and starts reseeding:

```
if exc.code in ("NoSuchKey", "NoSuchObject", "NoSuchBucket"):
    # MinIO without a persistent disk loses everything on restart: detect that and reseed.
    if await asyncio.to_thread(_storage_was_wiped):
        state.metrics.inc("reseeds")
        start_seeding()
        return JSONResponse({"error": "origin storage was reset, reseeding"}, status_code=503)
```

6. **Publish.** `POST /admin/publish` takes a lock, reads the index, generates a new breaking article with its two images, writes them and an updated index to MinIO, and returns the new article. The router then purges the cached index everywhere (Recipe 13).

Taste test: `http://localhost:8000/cache/status` shows `origin.seed` as `{"seeded": true, "objects": 184}` on a fresh start (60 articles × 3 objects + 3 static files + the index).

Recipe 11: The origin shield (`services/shield/shield_app.py`)

Goal: a shared second-level cache that protects the origin. It has three defences: request collapsing, a bounded pool of origin fetches, and a circuit breaker.

Method:

1. **Cache-aside on `GET /object/{id}`.** A fresh copy is returned as `HIT`. On a miss it fetches from the origin and stores the result for twice the edge TTL.

2. **Request collapsing.** Keep a dictionary of in-flight fetches, one future per object. The first request does the work; everyone arriving while it runs waits on the same future:

```python
async def collapsed_fetch(object_id: str) -> tuple[Fetched, bool]:
    """Ensure concurrent misses for the same object trigger exactly one origin fetch."""
    if not state.config.collapsing:
        return await fetch_from_origin(object_id), False
    pending = inflight.get(object_id)
    if pending is not None:
        state.metrics.inc("collapsed")
        return await asyncio.shield(pending), True
    future: asyncio.Future = asyncio.get_running_loop().create_future()
    inflight[object_id] = future
    try:
        result = await fetch_from_origin(object_id)
        future.set_result(result)
        return result, False
    except asyncio.CancelledError:
        future.cancel()
        raise
    except Exception as exc:
        future.set_exception(exc)
        future.exception()  # mark retrieved so lone failures don't log warnings
        raise
    finally:
        inflight.pop(object_id, None)
```

   Details that matter:

   - `asyncio.shield(pending)`: if one waiting client disconnects, the shared fetch isn't cancelled for everyone else.
   - A failure is shared too: all waiters get the same error, and there is still only one origin call.
   - `finally` always removes the entry, so the next miss after an error starts a fresh fetch.
   - Collapsing can be switched off at runtime, which the benchmark uses for its comparison.

3. **A bounded origin pool.** An `asyncio.Semaphore(4)` lets at most 4 fetches reach the origin at once; the rest queue. The pool's `waiting` and `active` counts are shown on the dashboard.
4. **A circuit breaker.** After 5 failures in a row the breaker *opens* and requests fail immediately for 10 s instead of piling onto a dead origin. Then it goes *half-open* and allows exactly one trial request. Success closes it again; failure reopens it:

```
def before_call(self) -> None:
    if self.state == "open":
        if time.monotonic() - self.opened_at < self.reset_seconds:
            raise CircuitOpen("origin circuit is open")
        self.state = "half-open"
    if self.state == "half-open":
        if self.trial_in_flight:
            raise CircuitOpen("origin circuit is half-open (trial in flight)")
        self.trial_in_flight = True

def record_failure(self) -> None:
    self.failures += 1
    self.trial_in_flight = False
    if self.state == "half-open" or (self.state == "closed" and self.failures >= self.threshold):
        self.state = "open"
        self.opened_at = time.monotonic()
        self.trips += 1
```

A 404 counts as a *success* for the breaker: the origin is healthy, the object just doesn't exist.

5. **Stale on error.** If the origin fails (or the breaker is open) and an expired copy is still in Redis, return it labelled `STALE`. Otherwise return 503.

Why it's built this way: the pool and breaker came from modelling the system in the Breakscale simulator, which showed the origin being overwhelmed during bursts. Collapsing removes duplicate work, the pool caps what's left, and the breaker stops hammering an origin that's already down.

Taste test: `test_request_collapsing_single_origin_fetch` fires 50 concurrent requests for one object against a fake slow origin and checks it was called exactly once, with 49 requests collapsed.

Recipe 12: The edge cache (`services/edge/edge_app.py`)

Goal: the first cache a user hits. It's fast when it has the object and resilient when it doesn't.

Method:

1. **Cache-aside on `GET /content/{id}`.** A fresh copy is returned as `HIT-EDGE`.
2. **On a miss, ask the shield** and translate its answer into the layer label (`HIT` → `HIT-SHIELD`, `MISS` → `MISS-ORIGIN`, `COLLAPSED` → `MISS-COLLAPSED`, `STALE` → `STALE-SHIELD`). The edge waits one second longer than the shield does, so the shield always answers or gives up first.
3. **Bypass a dead shield.** If the shield is unreachable (`node_down`), go straight to the origin and label it `MISS-ORIGIN-BYPASS`. Remember that for 2 seconds, so the next requests skip the dead shield instead of each paying the connection failure:

```
if time.monotonic() >= shield_down_until:
    try:
        fetched = await fetch(http, f"{config.SHIELD_URL}/object/{object_id}", timeout + 1)
        return fetched, SHIELD_TO_LAYER.get(fetched.headers.get("x-cache-shield", "MISS"), "MISS-ORIGIN")
    except UpstreamError as exc:
        if not exc.node_down:
            return None, str(exc)  # shield is alive but the origin failed and it had no stale copy
        shield_down_until = time.monotonic() + SHIELD_RETRY_SECONDS
# Shield unreachable: bypass it and go straight to the origin.
state.metrics.inc("shield_bypass")
```

A shield that is *alive* but reports an origin failure is not bypassed, since the origin is the problem.

4. **Store and serve.** Store the object for its class's TTL. A stale copy relayed by the shield is stored for only 5 s, so recovery is noticed quickly.
5. **Last resort.** If nothing upstream works and the edge has an expired copy, serve it as `STALE-EDGE`.
6. `POST /admin/shield-recovered` clears the 2-second skip window. The router calls it when the shield is restored, so the change takes effect immediately.
7. Every response carries `Cache-Control: no-store`, so the browser doesn't cache and every page view exercises the CDN in the demo.

Why it's built this way: the skip window and the recovery call were both added after testing. The first showed each request paying 4 s to rediscover a dead shield; the second showed edges still bypassing a shield that had already been restored.

Recipe 13: The router (`services/router/router_app.py`)

Goal: the public entry point. It load-balances, fails over, aggregates metrics and exposes the control API.

Method:

1. **Health checks.** A background task calls `GET /health` on every node every `HEALTH_INTERVAL_S` seconds and records up/down and latency.
2. **Round-robin over healthy edges, then the rest as fallbacks:**

```
def edge_order() -> list[str]:
    names = list(EDGES)
    healthy = [n for n in names if health[n]["up"]] or names
    start = next(rr) % len(healthy)
    ordered = healthy[start:] + healthy[:start]
    return ordered + [n for n in names if n not in ordered]
```

3. **Forward with failover.** Try each edge in that order:
   - Connection error, connect timeout, `503` with `X-Node-Down`, `502` or `504`: mark the edge down and try the next one.
   - **Read timeout:** return 504 *without* failing over, because a slow edge is usually waiting on a slow origin and another edge would just repeat the wait.
   - Anything else: done. Record the layer, latency and request in a 200-entry log, and add `X-Request-Id` and `X-Latency-ms`.
4. **Content API:** `GET /article/{id}` (shorthand for `article:{id}`), `GET /content/{object_id}`, and `GET /direct/{object_id}`, which skips the CDN entirely as a benchmark baseline.
5. **Metrics aggregation.** `GET /metrics` collects `/admin/metrics` from every node in parallel and computes the summary:

```
"edge_hit_ratio": ratio(edge_hits, total),
"cache_hit_ratio": ratio(edge_hits + shield_hits + stale, total),
"shield_hit_ratio": ratio(shield_hits, edge_misses),
"collapsing_efficiency": ratio(collapsed, collapsed + shield_fetches),
```

Collapsing efficiency is the share of shield misses that didn't need their own origin fetch.

6. **Control API:** `POST /cache/flush`, `POST /failure/{component}` (and `/failure/reset`), `GET/POST /config` (broadcast to every node), `POST /metrics/reset`, `POST /api/publish` and `POST /api/benchmark`.
7. **Publish and purge.** After the origin publishes, the router deletes `index:latest` from every edge and the shield, so the next homepage load fetches the new list.
8. **Protect the controls.** A middleware requires `X-Control-Token` on every POST when `CONTROL_TOKEN` is set. CORS allows the web service's origin and exposes the `X-Cache` headers to browser JavaScript, which the inspector needs:

```
app.add_middleware(ControlAuth)
app.add_middleware(
    CORSMiddleware,
    allow_origins=[o.strip() for o in config.WEB_ORIGINS.split(",") if o.strip()],
    allow_methods=["GET", "POST", "OPTIONS"],
    allow_headers=["content-type", "x-control-token"],
    expose_headers=["x-cache", "x-served-by", "x-content-class", "x-latency-ms",
                    "x-request-id", "x-edge-attempts"],
)
```

   CORS is added last, so it wraps everything and answers browser preflight requests before the token check.
9. **Liveness.** `GET /healthz` always returns 200. `GET /health` reports `ok`, `degraded` or `down` for the whole system.

Why it's built this way: the router doesn't use `install_admin`, so it needs its own `/healthz`. The first Render deploy hung for 15 minutes because the Blueprint pointed the router's health check at `/healthz` and the router didn't have one.

Taste test:

```
curl -i http://localhost:8000/article/world-013
```

Repeat it five times. You'll see `MISS-ORIGIN`, then `HIT-SHIELD` on the other two edges, then `HIT-EDGE`, with `X-Served-By` rotating through edge-1, edge-2 and edge-3.

Recipe 14: The web pages (`services/web` and `content/static/app.js`)

Goal: a newspaper that visibly runs on the CDN, and a control room to watch and break it.

The newspaper

1. `index.html` is a tiny shell. It loads `config.js`, which says where the router is (`window.CDN_API = "http://localhost:8000"`), then loads the stylesheet, logo and `app.js` **through the CDN** as `static:` objects.
2. `app.js` fetches every object with `fetch()` rather than plain `<img>` tags, so it can read the response headers:

```
async function cdn(path, as) {
  const started = performance.now();
  const resp = await fetch(API + path, { cache: "no-store" });
  const entry = {
    path: path.replace("/content/", "").replace("/article/", "article:"),
    layer: resp.headers.get("X-Cache") || (resp.ok ? "?" : "ERROR"),
    edge: resp.headers.get("X-Served-By") || "-",
    ms: Number(resp.headers.get("X-Latency-ms")) || performance.now() - started,
    status: resp.status,
  };
  // ... added to the inspector log
}
```

Images are fetched as blobs and shown with `URL.createObjectURL()`.

3. Navigation uses the URL hash (`#/`, `#/section/sports`, `#/article/world-013`), so no server routing is needed.
4. The **CDN inspector** panel lists recent objects with their layer, edge and latency. "Toggle image badges" overlays the same information on every picture.

The dashboard

1. It polls `GET /metrics` every second and `GET /api/requests` and `GET /cache/status` every 2.5 seconds.
2. The topology panel draws one card per node with live stats and a **Fail/Restore** button.
3. KPI tiles, a stacked bar of serving layers, and two line charts (throughput and p95 latency over the last 60 seconds) with hover tooltips. They're drawn as plain SVG, with no chart library.
4. **Traffic generator:** every 100 ms it sends `rps/10` requests from the browser, chosen by Zipf popularity, a single hot article, or uniformly. At most 150 are in flight at once.
5. Controls for collapsing, origin delay and TTLs, flushes, publishing, restoring and resetting.
6. If a control call returns 401, the page asks for the control token once and keeps it in `sessionStorage` for that tab.

Why it's built this way: the pages live in their own static service because a static site is free on Render and doesn't use instance hours. That's also why the router needs CORS.

Recipe 15: Benchmarks (`services/router/bench.py`)

Goal: reproduce the project document's comparisons with real measurements.

Method:

1. A `Control` helper wraps the router's API (`flush`, `reset`, `configure`, `fail`, `fire`).
2. `fire(paths, concurrency)` sends requests with a concurrency limit and summarises success rate, avg/p50/p95/p99 latency, throughput, hit ratio and the count of each `X-Cache` label.
3. A Zipf generator makes realistic traffic, where a few hot objects get most of the requests:

```
def zipf_paths(objects, count, s=1.1, seed=7):
    rng = random.Random(seed)
    weights = [1 / (rank + 1) ** s for rank in range(len(objects))]
    return [f"/content/{o}" for o in rng.choices(objects, weights=weights, k=count)]
```

4. Six scenarios:

Scenario	What it does
`workload`	1,500 Zipf requests on cold caches
`direct_vs_edge`	The same hot articles via `/direct` vs via warm edges
`shield_vs_origin`	Cold fetch, then flush edges and fetch from the shield, then edge hits
`collapsing`	200 simultaneous requests for one uncached image, collapsing off then on
`delayed_origin`	Cold and warm latency with 0 ms vs 300 ms origin delay
`failures`	Edge-1 down, shield down, origin down, then origin down with every cache expired

5. The last failure phase sets every TTL to 2 seconds, warms the caches, waits for everything to expire and stops the origin. Only stale copies can then be served, which proves stale serving and the circuit breaker together.
6. `run()` saves the configuration first and restores it, plus all components, at the end, even if a scenario fails.
7. The router runs benchmarks as a background task (`POST /api/benchmark`); the dashboard polls `GET /api/benchmark` for progress and results. There's also a command-line entry point.

Results measured locally (Docker Compose, no CPU limits, 120 ms origin delay):

Comparison	Result
Direct origin vs edge cache	142.6 ms → 6.0 ms average (24×), 135 → 3,050 requests/s
Origin fetch vs shield hit vs edge hit	324 ms → 5.1 ms → 4.2 ms average
200 simultaneous requests, without vs with collapsing	origin requests 172 → 1; p95 5,853 ms → 196 ms
Normal vs 300 ms origin	cold 10 ms vs 643 ms; warm about 3 ms either way
Zipf workload, cold caches	81.8% cache hit ratio
All failure phases	100% success

Part 4. Packaging and serving

Recipe 16: Dockerfiles and start scripts

Goal: one small image per service, built from the repository root so it can include `shared/`.

Method:

1. Every Python service follows the same pattern: slim base image, install only its own requirements, copy `cdncore` and its own code, run as a non-root user. The edge:

```
FROM python:3.12-slim
RUN apt-get update \
    && apt-get install -y --no-install-recommends redis-server \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY services/edge/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY shared/cdncore ./cdncore
COPY shared/start-with-redis.sh ./start.sh
COPY services/edge/edge_app.py .
RUN useradd -m -u 1000 app
USER app
ENV PYTHONUNBUFFERED=1
CMD ["sh", "start.sh", "edge_app:app"]
```

2. **Cache services start their own Redis when none is given.** `start-with-redis.sh`:

```
if [ -z "$REDIS_URL" ]; then
  redis-server --port 6379 --bind 127.0.0.1 --dir /tmp --save "" --appendonly no \
    --maxmemory "${REDIS_MAXMEMORY:-64mb}" --maxmemory-policy allkeys-lru --daemonize yes
  export REDIS_URL=redis://127.0.0.1:6379/0
fi
exec uvicorn "$1" --host 0.0.0.0 --port "${PORT:-8000}" --no-access-log --log-level warning
```

3. The origin image also installs `fonts-dejavu-core` (for image labels) and pre-renders the images with `RUN python -c "from content import generator; generator.prerender_images()"`.
4. Every service listens on `${PORT:-8000}`, so hosts that assign a port (Render uses 10000) work unchanged.
5. Access logs are off (`--no-access-log`) because logging every request slows benchmarks down.

Why it's built this way: in Docker Compose each cache gets a separate Redis container, which is the textbook layout. On Render's free plan only one managed Redis is allowed per workspace, so each cache runs a private Redis inside its own container instead. The same image does both, depending on whether `REDIS_URL` is set.

Recipe 17: Run everything with Docker Compose

Goal: start all 12 containers with one command.

Method:

1. `docker-compose.yml` defines MinIO, the origin, the shield and its Redis, three edges each with a Redis, the router, and an `nginx` container serving `services/web`.

2. Services find each other by name (`http://shield:8000`) on Compose's internal network. Only the router (8000), the web pages (8080) and the MinIO console (9001) are published to your machine.
3. Start it:

```
docker compose up --build -d
```

4. Open `http://localhost:8080` (newspaper), `http://localhost:8080/dashboard.html`, `http://localhost:8000/docs` (API) and `http://localhost:9001` (MinIO console).

Simulating Render's free plan locally: `docker-compose.render-sim.yml` is layered on top. It limits every service to 512 MB and 0.1 CPU, sets tokens and Render's longer timeouts, removes the separate Redis URLs (so each cache starts its own), and gives MinIO no persistent volume:

```
docker compose -f docker-compose.yml -f docker-compose.render-sim.yml up --build -d
```

Taste test: `docker compose ps` shows 12 running containers, and `http://localhost:8000/health` shows `"status": "ok"` with `"healthy_edges": 3`.

Recipe 18: Tests

Goal: prove the hard parts work, both in isolation and against the running system.

Method:

1. `tests/conftest.py` adds `shared/` and each service folder to Python's import path, so the tests can import `shield_app`, `cdncore` and `content` directly.
2. **Unit tests** (`test_units.py`, 9 tests, no infrastructure needed): edge-list parsing, TTL classes, config booleans, percentiles, generator determinism, the circuit breaker's state machine, collapsing with one and with many callers, collapsing switched off, and a failure shared by all waiters.
3. **End-to-end tests** (`test_e2e.py`, 6 tests, skipped unless `CDN_URL` is set): the tiered path, edge failover, shield bypass, serving cached content while the origin is down (and 503 for uncached content), publish-and-purge, and the shape of `/metrics` and `/health`.
4. Run all 15 in a temporary container on the Compose network:

```
docker run --rm --network miniproject_default -e CDN_URL=http://router:8000 -v "%cd%":/src -w /src
python:3.12-slim sh -c "pip install -q -r requirements-dev.txt && python -m pytest -v tests"
```

   In PowerShell, replace `"%cd%"` with `"${PWD}"`. You should see `15 passed`.

Part 5. Serving it to the world

Recipe 19: Deploy on Render

Goal: each microservice as its own Render service on the free plan.

Method (summary): the full settings, variables and troubleshooting are in `docs/RENDER_DEPLOYMENT.md`. The key points:

1. `render.yaml` is a Blueprint that creates seven Docker web services (minio, origin, shield, edge-1/2/3, router) and one static site (web), plus an environment group `edge-cdn-shared` with the shared token, MinIO keys and timeouts.
2. Free web services can't *receive* private-network traffic, so services call each other through their public `https://<name>.onrender.com` URLs. That's why the internal token exists.
3. URLs are entered when the Blueprint is applied. Use the predicted `https://<service-name>.onrender.com` values, then correct any that got a random suffix.
4. Health checks use `/healthz`, never `/health` (see Recipe 6).
5. Timeouts are raised (connect 15 s, upstream 90 s) because a sleeping service takes about a minute to wake.
6. The static site writes the router's URL into `config.js` at build time: `printf 'window.CDN_API = "%s";' "$CDN_API_URL" > config.js`.

Free-plan limits to expect: services sleep after 15 minutes idle; 750 instance hours a month are shared across all services; there are no disks, so MinIO's data (and published articles) disappear when it sleeps and the origin reseeds; bandwidth is limited; and performance at 0.1 CPU is far below the local numbers.

Part 6. Lessons from the kitchen

These problems came up while building the project. Each one shaped a part of the final design.

Problem	Symptom	Fix
Official MinIO images withdrawn	`docker pull minio/minio` failed	Use the `pgsty/minio` community build
httpx too slow	Router capped at about 390 req/s while an edge did 1,650	Use aiohttp for every hop (about 1,860 req/s)
Starlette `BaseHTTPMiddleware` overhead	Extra cost on every request	Pure ASGI middleware classes
Timeouts treated as dead nodes	MinIO stopped, so the router marked all healthy edges dead	Separate connection errors from read timeouts; MinIO client fails fast
Slow DNS failure for stopped containers	First failover took 7.9 s	1.5 s connect timeout (now about 1.5 s)
Shield rediscovered as dead on every request	Each bypass cost about 4 s	2-second skip window at each edge
Edges still bypassing a restored shield	Spurious bypasses after a restore	Router calls `/admin/shield-recovered`
Docker kept MinIO's anonymous volume	A "wipe" test didn't actually wipe	`docker compose up --force-recreate -V` to really test reseeding
Free Docker hosting on Hugging Face ended	`402 Payment Required`	Move to Render, one service per microservice
Simulated failures would restart services on Render	Health check would fail during a demo	Separate `/healthz` (liveness) from `/health`
Router had no `/healthz`	First Render deploy hung for 15 minutes	Add `/healthz` to the router
CRLF line endings on Windows	Shell scripts could break in Linux containers	`.gitattributes` with `eol=lf`
Rendering images at startup	Minutes of startup on a 0.1-CPU instance	Pre-render at Docker build time

Ideas for a second edition

- Persist logs and metrics outside memory, so a restart doesn't wipe them.
- Structured per-request logs across all hops, joined by `X-Request-Id`.
- Consistent hashing instead of round-robin, so each object lives on one edge and the hit ratio rises.
- Prometheus metrics and Grafana dashboards.
- Collapsing at the edges as well as at the shield.